

Лекція

**Блокуюча передача повідомлень
в MPI та реалізація паралельного
алгоритму множення матриць**

Розрізняють:

- ✓ блокуючу передачу повідомлень,
- ✓ неблокуючу передачу повідомлень.

Передача повідомлень в MPI

- Передача повідомлення відбувається від процесу-відправника до процесу-отримувача по каналу зв'язку. Будь-яке повідомлення містить набір **однотипних** даних.
- Методи відправки та прийому повідомлень MPI оперують масивами однотипних елементів, які називають **буферами**. Розмір буферів вказується не в байтах, а в елементах.
- Програміст має забезпечити, щоб фактичний тип змінних та тип, вказаний в аргументах функції MPI, збігались. Компілятор такої перевірки **не виконує та не видає попереджень**, якщо в аргументах функції MPI вказано тип, що не відповідає типу змінної.
- Програміст має забезпечити відповідність розміру буфера отримувача кількості даних, що надходять від відправника.

Передача повідомлень в MPI

Передача повідомлення відбувається в три кроки:

- 1) відправка даних процесом-відправником (зчитування даних з буфера у програмі процесу-відправника та формування повідомлення),
- 2) розміщення в буфері повідомлень (на стороні відправника або отримувача),
- 3) отримання повідомлення процесом-отримувачем (пошук повідомлення у буфері повідомлень та записування даних з повідомлення у буфер програми процесу-отримувача).

Процес-відправник, в залежності від обраного методу передачі повідомлення, може або одразу відправити дані або очікувати на сигнал готовності від процесу-отримувача. Процес-отримувач, в залежності від обраного методу передачі повідомлення, може або одразу виконувати отримання даних з повідомлення (якщо воно знайдено у буфері повідомлень) або подавати сигнал готовності процесу відправнику і очікувати на надходження повідомлення.

Якщо на момент отримання повідомлення його немає в буфері повідомлень процесу-отримувача, то процес-отримувач очікуватиме на його надходження. Пошук повідомлення у буфері повідомлень здійснюється за вказівкою на процес-відправник, від якого очікують передачу даних, за типом даних та їх кількістю. Використовують також допоміжні ідентифікатори повідомлення, такі як тег.

Передача повідомлень в MPI

- Тег – це ціле число від 0 до 32767, що використовують для того, щоб ідентифікувати повідомлення. Відправник встановлює тег повідомлення, а отримувач може обрати такий режим прийому, щоб отримувати тільки повідомлення з заданим тегом. MPI_ANY_TAG означає прийом повідомлення з будь-яким тегом.
- MPI_ANY_SOURCE означає прийом повідомлення від будь-якого процесу.
- Статус – посилання на структуру MPI_STATUS, в яку буде записано інформацію про прийняте повідомлення. Структура MPI_STATUS містить такі поля: MPI_SOURCE, MPI_TAG, MPI_ERROR. Інформація про повідомлення може бути потрібна програмі, якщо під час прийому використовувались MPI_ANY_TAG або MPI_ANY_SOURCE, тобто отримувач не знає відправника або тегу повідомлення. MPI_STATUS_IGNORE означає, що програмі інформація про повідомлення не потрібна.

Блокуючий обмін повідомленнями

- Метод `MPI_Send` повертає управління програмі тоді, коли всі дані, вказані методом для відправки, скопійовані у буфер повідомлень на стороні процесу-отримувача.
- В момент, коли метод `MPI_Send` повертає управління програмі, не гарантується, що повідомлення вже отримано отримувачем.
- Метод `MPI_Recv` обирає повідомлення за вказаними аргументами: ранг відправника, тег, комунікатор. Якщо відповідність встановлена, то повідомлення приймається повністю, а функція `MPI_Recv` повертає управління програмі.
- Коли `MPI_Recv` повернула управління програмі, гарантується, що всі дані, які отримуються, повністю скопійовані з каналу зв'язку в буфер отримувача.

Point to Point Communication: blocking send/receive

```
int MPI_Send(    void* buf,  
                int count,  
                MPI_Datatype datatype,  
                int dest,  
                int tag,  
                MPI_Comm comm    );
```

```
int MPI_Recv(    void* buf,  
                int count,  
                MPI_Datatype datatype,  
                int source,  
                int tag,  
                MPI_Comm comm,  
                MPI_Status *status    );
```

Приклад: тестування передачі повідомлень із заданим тегом

```
#include <stdio.h>
#include <mpi.h>

const int TAG_DATA1 = 10;
const int TAG_DATA2 = 20;
const int TAG_DATA3 = 25;

int main(int argc, char *argv[]) {
    MPI_Init(&argc, &argv);

    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if(rank == 0) {
        int x;
        MPI_Recv(&x, 1, MPI_INT, MPI_ANY_SOURCE, TAG_DATA3, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Got %d with tag TAG_DATA3\n", x);
        MPI_Recv(&x, 1, MPI_INT, MPI_ANY_SOURCE, TAG_DATA1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Got %d with tag TAG_DATA1\n", x);
        MPI_Recv(&x, 1, MPI_INT, MPI_ANY_SOURCE, TAG_DATA2, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Got %d with tag TAG_DATA2\n", x);
    }
    else if(rank == 1) {
        int x = 1000;
        MPI_Send(&x, 1, MPI_INT, 0, TAG_DATA1, MPI_COMM_WORLD);
    }
    else if(rank == 2) {
        int x = 2000;
        MPI_Send(&x, 1, MPI_INT, 0, TAG_DATA2, MPI_COMM_WORLD);
    }
    else {
        int x = 3000;
        MPI_Send(&x, 1, MPI_INT, 0, TAG_DATA3, MPI_COMM_WORLD);
    }
    MPI_Finalize();
    return 0;
}
```

Паралельне обчислення суми ряду

за Стіренко С.Г. «Засоби паралельного програмування»

Нехай є **p** процесів.

Задача з номером **id** буде обчислювати члени ряду з номерами **id**, **p + id**, **2p + id**, ...

$$f(x_0) = \underbrace{a_1 + a_2 + \dots + a_p}_{\text{крок 1}} + \underbrace{a_{p+1} + a_{p+2} + \dots + a_{2p}}_{\text{крок 2}} + \dots$$

```
#include <mpi.h>
#include <stdlib.h>
#include <stdio.h>
#include <stdbool.h>
#include <math.h>

const double EPSILON = 1E-100; // Точність обчислення суми ряду
const int VALUE_TAG = 1; // Теґ показника ступеня числа Е
const int NUM_TAG = 2; // Теґ номера поточного члену ряду
const int TERM_TAG = 3; // Теґ значення поточного члену ряду
const int BREAK_TAG = 4; // Теґ завершення обчислень
const char *input_file_name = "in.txt";
const char *output_file_name = "out.txt";

double factorial(int value) { ...}

double calc_series_term(int term_number, double value) { ...}
```

Паралельне обчислення суми ряду

```
int main(int argc, char *argv[]) {
    MPI_Init(&argc, &argv);
    int rank; /*id процесу*/
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    int p; /* кількість процесів*/
    MPI_Comm_size(MPI_COMM_WORLD, &p);
    double parametr; /* параметр суми ряду */
    if(rank == 0) {/*MASTER*/
        FILE *input_file = fopen(input_file_name, "r");
        if(!input_file) {
            fprintf(stderr, "Can't open input file!\n\n");
            MPI_Abort(MPI_COMM_WORLD, 1);
            return 1;
        }
        fscanf(input_file, "%lf", &parametr);
        fclose(input_file);

        for(int id = 1; id < p; id++) {
            MPI_Send(&parametr, 1, MPI_DOUBLE, id, VALUE_TAG, MPI_COMM_WORLD);
        }
    }
    else {/*WORKER*/
        MPI_Recv(&parametr, 1, MPI_DOUBLE, 0, VALUE_TAG, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    }
}
```

Паралельне обчислення суми ряду

```
int last_num = 0;    /*номер останнього обчисленого члену ряду */
double sum = 0.0;    /*сума членів ряду */

for(int step = 0; step < 1000; step++){
    int num; /* номер члену ряду, що обчислюється */
    if(rank == 0) { /*MASTER*/
        num = last_num++;
        int curr_num = last_num;
        for(int id = 1; id < p; id++) {
            MPI_Send(&curr_num, 1, MPI_INT, id, NUM_TAG, MPI_COMM_WORLD);
            curr_num++;
        }
        last_num = curr_num;
    }
    else{ /*WORKER*/
        MPI_Recv(&num, 1, MPI_INT, 0, NUM_TAG, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    }

    double term = calc_series_term(num, parametr);
    int need_break = false; /* умова «досягнуто необхідну точність» */
}
```

Паралельне обчислення суми ряду

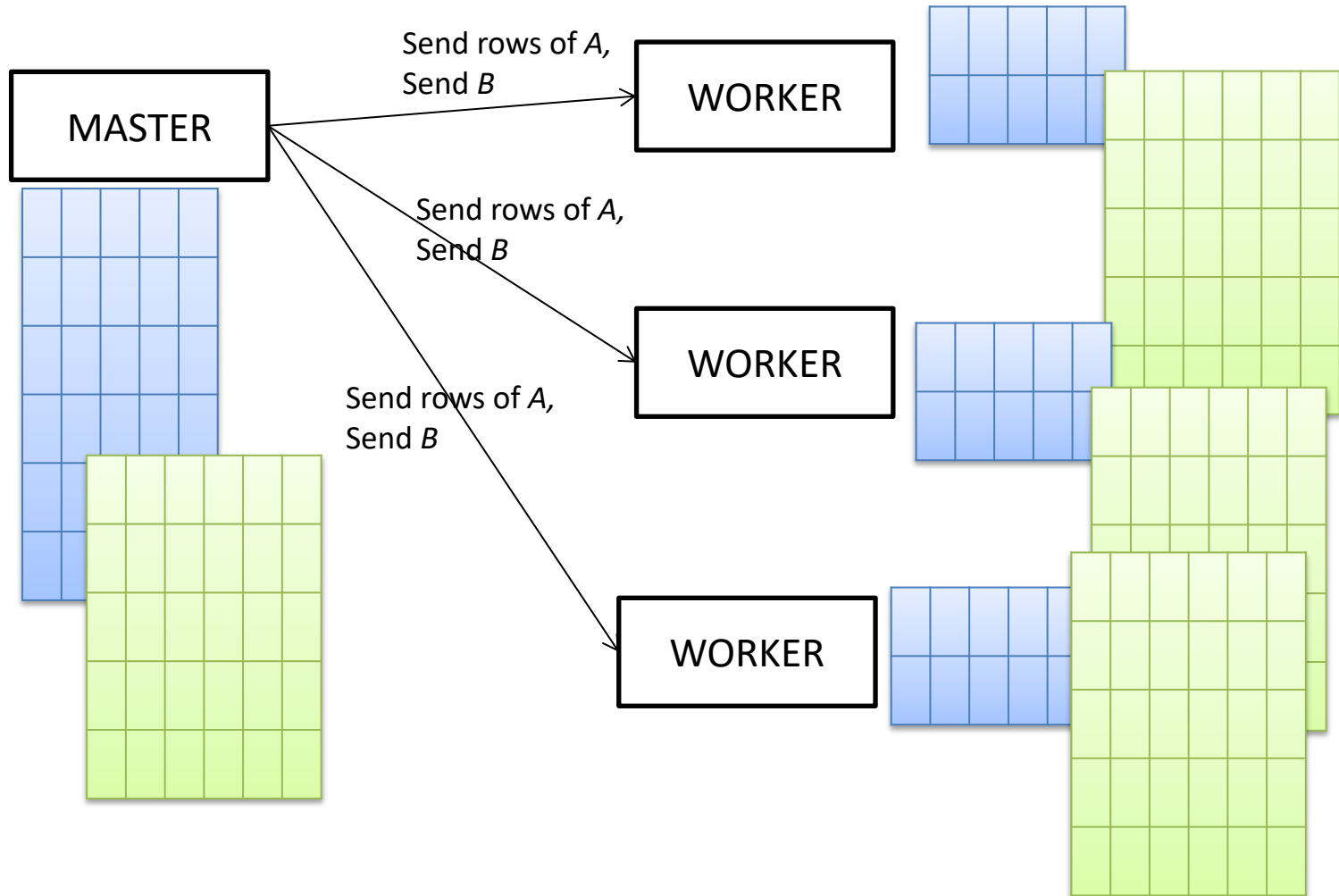
```
if(rank == 0) { /*MASTER*/
    double curr_term = term;
    sum += curr_term;
    if(curr_term < EPSILON) {
        need_break = true;
    }
    for(int id = 1; id < p; id++) {
        MPI_Recv(&curr_term, 1, MPI_DOUBLE, id, TERM_TAG, MPI_COMM_WORLD, MPI_STATUS_
        sum += curr_term;
        if(curr_term < EPSILON) {
            need_break = true;
        }
        MPI_Send(&need_break, 1, MPI_INT, id, BREAK_TAG, MPI_COMM_WORLD);
    }
}
else { /*WORKER*/
    MPI_Send(&term, 1, MPI_DOUBLE, 0, TERM_TAG, MPI_COMM_WORLD);
    MPI_Recv(&need_break, 1, MPI_INT, 0, BREAK_TAG, MPI_COMM_WORLD, MPI_STATUS_IGNORE)
}
if(need_break) {
    break;
}
} /* завершення циклу for step*/
```

Паралельне обчислення суми ряду

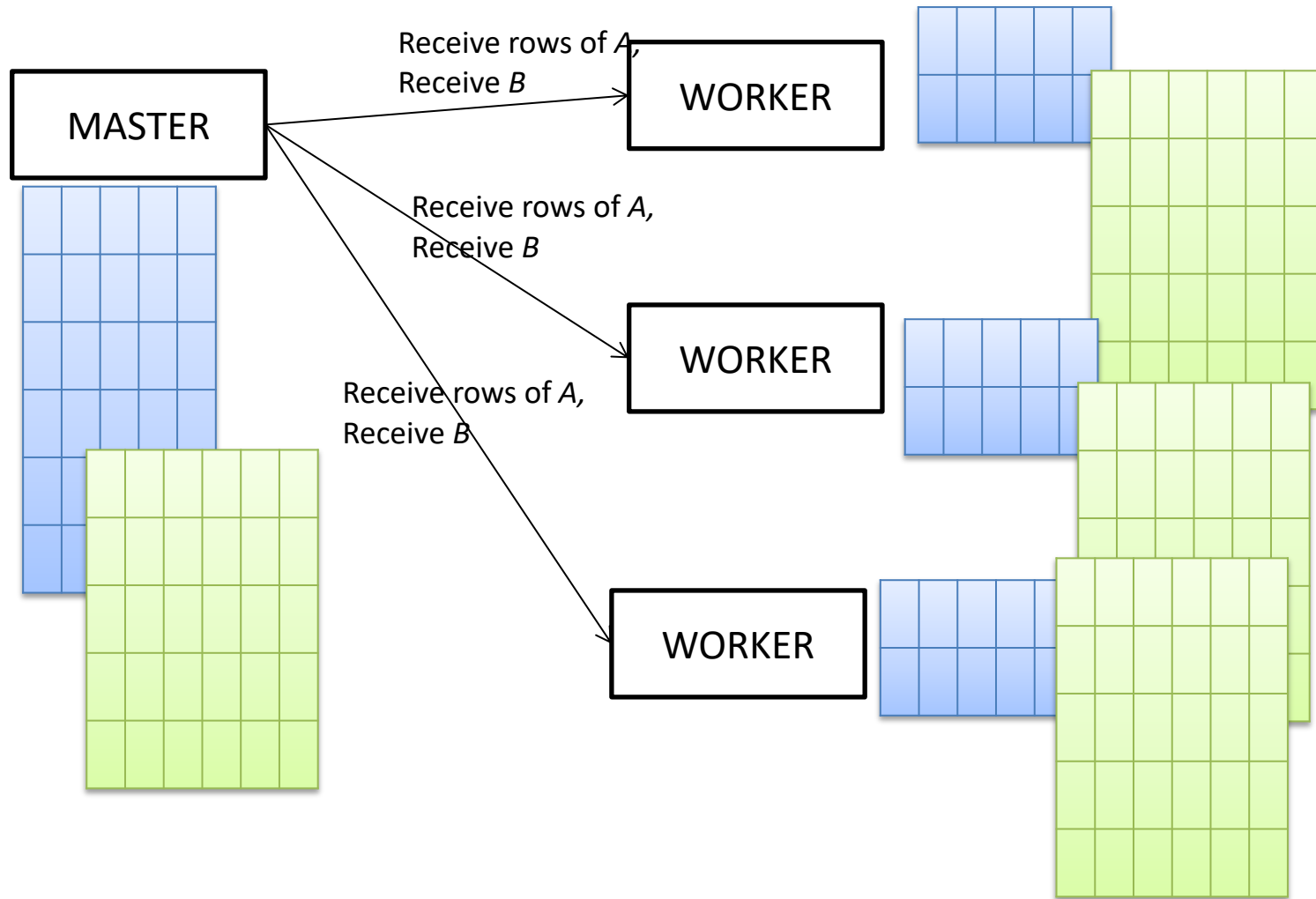
```
if(rank == 0) {  
    FILE *output_file = fopen(output_file_name, "w");  
    if(!output_file) {  
        fprintf(stderr, "Can't open output file!\n\n");  
        MPI_Abort(MPI_COMM_WORLD, 2);  
        return 2;  
    }  
    fprintf(output_file, "%.15lf\n", sum);  
}  
MPI_Finalize();  
return 0;  
}
```

Matrix multiply

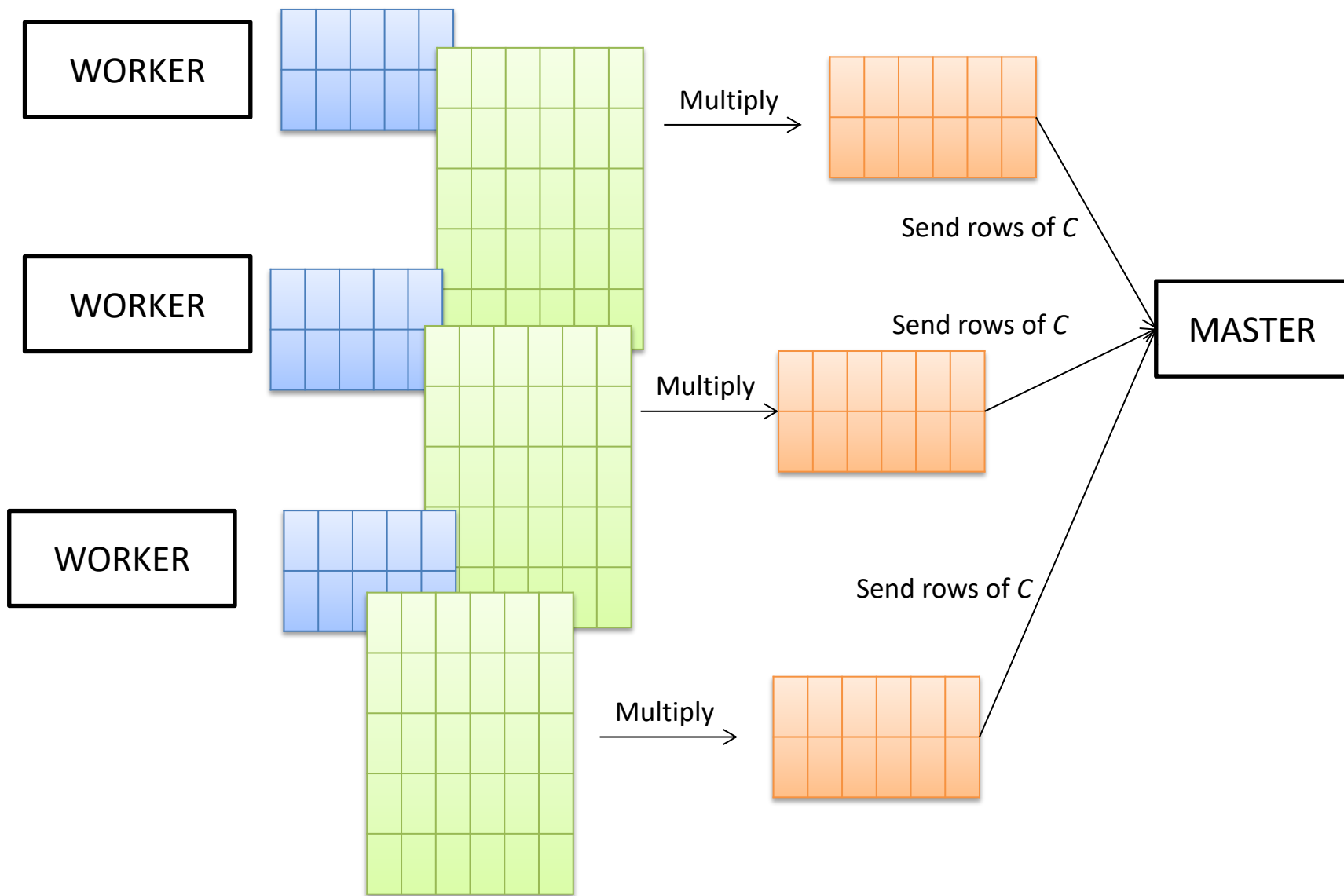
MASTER does:



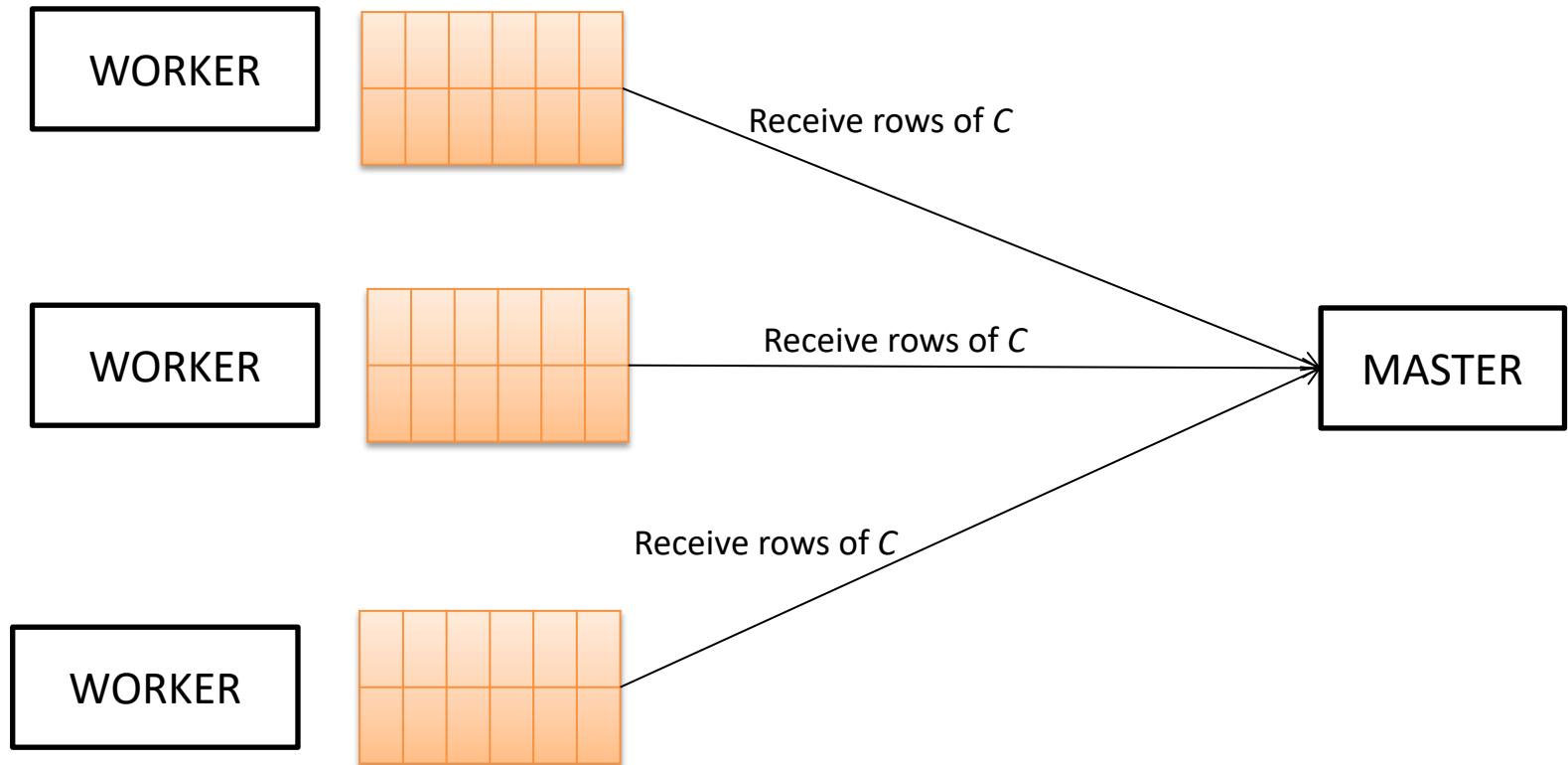
WORKERS do:



WORKERS do:



MASTER does:



Matrix multiply

```
/* FILE: mpi_mm.c  DESCRIPTION: MPI Matrix Multiply - C Version
* In this code, the master task distributes a matrix multiply
* operation to numtasks-1 worker tasks.
* AUTHOR: Blaise Barney.
* Adapted from Ros Leibensperger, Cornell Theory
*/

#include "mpi.h"
#include <stdio.h>
#include <stdlib.h>

#define NRA 62                /* number of rows in matrix A */
#define NCA 15                /* number of columns in matrix A */
#define NCB 7                 /* number of columns in matrix B */
#define MASTER 0              /* taskid of first task */
#define FROM_MASTER 1 /* setting a message type */
#define FROM_WORKER 2 /* setting a message type */
int main (int argc, char *argv[]) {
    int numtasks,
        taskid,
        numworkers,
        source,
        dest,
        mtype,                /* message type */
        rows,                 /* rows of matrix A sent to each worker */
        averow, extra, offset, /* used to determine rows sent to each worker*/
        i, j, k, rc;
    double a[NRA][NCA], /* matrix A to be multiplied */
           b[NCA][NCB], /* matrix B to be multiplied */
           c[NRA][NCB]; /* result matrix C */
    MPI_Status status;
    MPI_Init( &argc, &argv);
    MPI_Comm_size( MPI_COMM_WORLD, &numtasks);
    MPI_Comm_rank( MPI_COMM_WORLD, &taskid);
```

```

if (numtasks < 2 ) {
    printf("Need at least two MPI tasks. Quitting...\n");
    MPI_Abort(MPI_COMM_WORLD, rc);
    exit(1);
}
numworkers = numtasks-1;
if (taskid == MASTER) {
    printf("mpi_mm has started with %d tasks.\n", numtasks);
    for (i=0; i<NRA; i++)
        for (j=0; j<NCA; j++)
            a[i][j]= 10;
    for (i=0; i<NCA; i++)
        for (j=0; j<NCB; j++)
            b[i][j]= 10;

    averow = NRA/numworkers;
    extra = NRA%numworkers;
    offset = 0;
    for (dest=1; dest<=numworkers; dest++) {
        rows = (dest <= extra) ? averow+1 : averow;
        printf("Sending %d rows to task %d offset=%d\n", rows, dest, offset);
        MPI_Send(&offset, 1, MPI_INT, dest, FROM_MASTER, MPI_COMM_WORLD);
        MPI_Send(&rows, 1, MPI_INT, dest, FROM_MASTER, MPI_COMM_WORLD);
        MPI_Send(&a[offset][0], rows*NCA, MPI_DOUBLE, dest, FROM_MASTER,
                                                    MPI_COMM_WORLD);
        MPI_Send(&b, NCA*NCB, MPI_DOUBLE, dest, FROM_MASTER, MPI_COMM_WORLD);
        offset = offset + rows;
    }
}

```

```

/* Receive results from worker tasks */
for (source=1; source<=numworkers; source++) {
    MPI_Recv(&offset, 1, MPI_INT, source, FROM_WORKER,
            MPI_COMM_WORLD, &status);
    MPI_Recv(&rows, 1, MPI_INT, source, FROM_WORKER,
            MPI_COMM_WORLD, &status);
    MPI_Recv(&c[offset][0], rows*NCB, MPI_DOUBLE, source, FROM_WORKER,
            MPI_COMM_WORLD, &status);
    printf("Received results from task %d\n", id);
}
/* Print results */
printf("****\n");
printf("Result Matrix:\n");
for (i=0; i<NRA; i++) {
    printf("\n");
    for (j=0; j<NCB; j++)
        printf("%6.2f ", c[i][j]);
}
printf("\n*****\n");
printf ("Done.\n");
}

```

```

/***** worker task *****/
else{ /* if (taskid > MASTER) */
    MPI_Recv(&offset, 1, MPI_INT, MASTER, FROM_MASTER, MPI_COMM_WORLD,
                                                    &status);
    MPI_Recv(&rows, 1, MPI_INT, MASTER, FROM_MASTER, MPI_COMM_WORLD, &status);
    MPI_Recv(&a, rows*NCA, MPI_DOUBLE, MASTER, FROM_MASTER, MPI_COMM_WORLD,
                                                    &status);
    MPI_Recv(&b, NCA*NCB, MPI_DOUBLE, MASTER, FROM_MASTER, MPI_COMM_WORLD,
                                                    &status);

    for (k=0; k<NCB; k++)
        for (i=0; i<rows; i++) {
            c[i][k] = 0.0;
            for (j=0; j<NCA; j++)
                c[i][k] = c[i][k] + a[i][j] * b[j][k];
        }

    MPI_Send(&offset, 1, MPI_INT, MASTER, FROM_WORKER, MPI_COMM_WORLD);
    MPI_Send(&rows, 1, MPI_INT, MASTER, FROM_WORKER, MPI_COMM_WORLD);
    MPI_Send(&c, rows*NCB, MPI_DOUBLE, MASTER, FROM_WORKER, MPI_COMM_WORLD);
}
MPI_Finalize();
}

```